MCP-Lock

A lockfile and launch gate that stops AI coding agents from silently running new, unreviewed tool servers



Aakash G S · Hariharan J · J Jaswanth Raj

Dept. of Computer Science and Engineering (Data Science), SRM Institute of Science and Technology

The problem: every launch can run code nobody reviewed

1 Config says

`npx -y weather-mcp`

No version: “give me the latest”

2 AI agent

starts the server

Happens automatically, every session

3 npm / PyPI

serves newest version

May be hijacked or changed since review

4 Server runs

+ tools enter the model

Tool descriptions are read as instructions

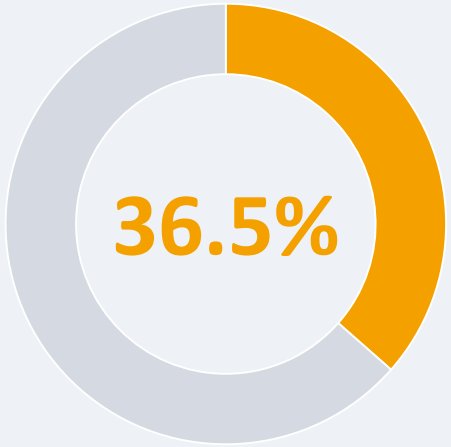
! New code runs on your machine

Install scripts and payloads execute with your files, keys and tokens. Hijacked maintainer accounts did exactly this in the 2025 Shai-Hulud worm waves.

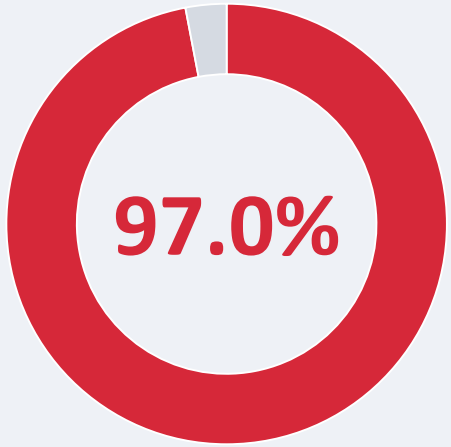
! New tool descriptions steer the agent

A “rug pull” swaps a harmless tool description for a poisoned one. The model obeys it, and no one sees the change.

How often does it happen? Our census of MCP packages



of 2,000 sampled MCP packages changed the tools they advertise within 90 days



of those changes carried no provenance, publisher or install-script signal: invisible to metadata checks

15,272

MCP packages on npm and PyPI in the metadata census

What this means

Checking names, ages or signatures is not enough. You have to compare the tool definitions themselves.

Source: RQ1 census, tool definitions statically extracted from package archives (stratified sample of 2,000).

What existing defences miss

Approach	Checks before launch, runs no package code	Pins tool definitions	Checks provenance, publisher and install scripts	No change to servers or registries
Pre-install hook (Bagmar & Saraf) — our baseline	✓	✗	✗	✓
Tool pinning: MCP-Scan, mcp-pin, mcpseal	✗	✓	partly	✓
Signed definitions: ETDI, attested admission	–	✓	–	✗
MCP-Lock (ours)	✓	✓	✓	✓

The gap: Kraishan measured drift in MCP registry fields and recommended client-side pinning of tool definitions. MCP-Lock is that pinning, combined with build-identity checks, and tested on real attacks.

"partly": one pinning proxy also records the package version and registry digest. Based on the Related Work section of our paper.

MCP-Lock: lock what you reviewed, check every launch

1 `mcplock lock`

Once, after you review a server. Writes mcp-lock.json, which records:

- Archive digest (exact bytes)
- Build identity (who built it, from where)
- Publisher account
- Install-time scripts
- Tool definitions (canonical hash)

2 Gate on every launch

Before the agent starts a server, the gate compares the version it would get with the lock, reading tools from the archive. It never executes package code.

ALLOW	matches the lock
WARN	changed tools: shows the diff for review
BLOCK	build, scripts or name look wrong

On a block it suggests a safe version the agent can retry with.

Five checks, four of them new

S1

Baseline checks

Name looks like a popular one, does not exist or is under 30 days old; untrusted source; known-vulnerable pin

BLOCK

S2

Provenance continuity

Build provenance disappears, or a different repository, workflow or builder built the new version

BLOCK

S3

Publisher

Published by hand after CI releases, or by someone who is not a maintainer

WARN

S4

Install scripts

A new install-time script appears (PyPI: source-only release after wheels)

BLOCK

S5

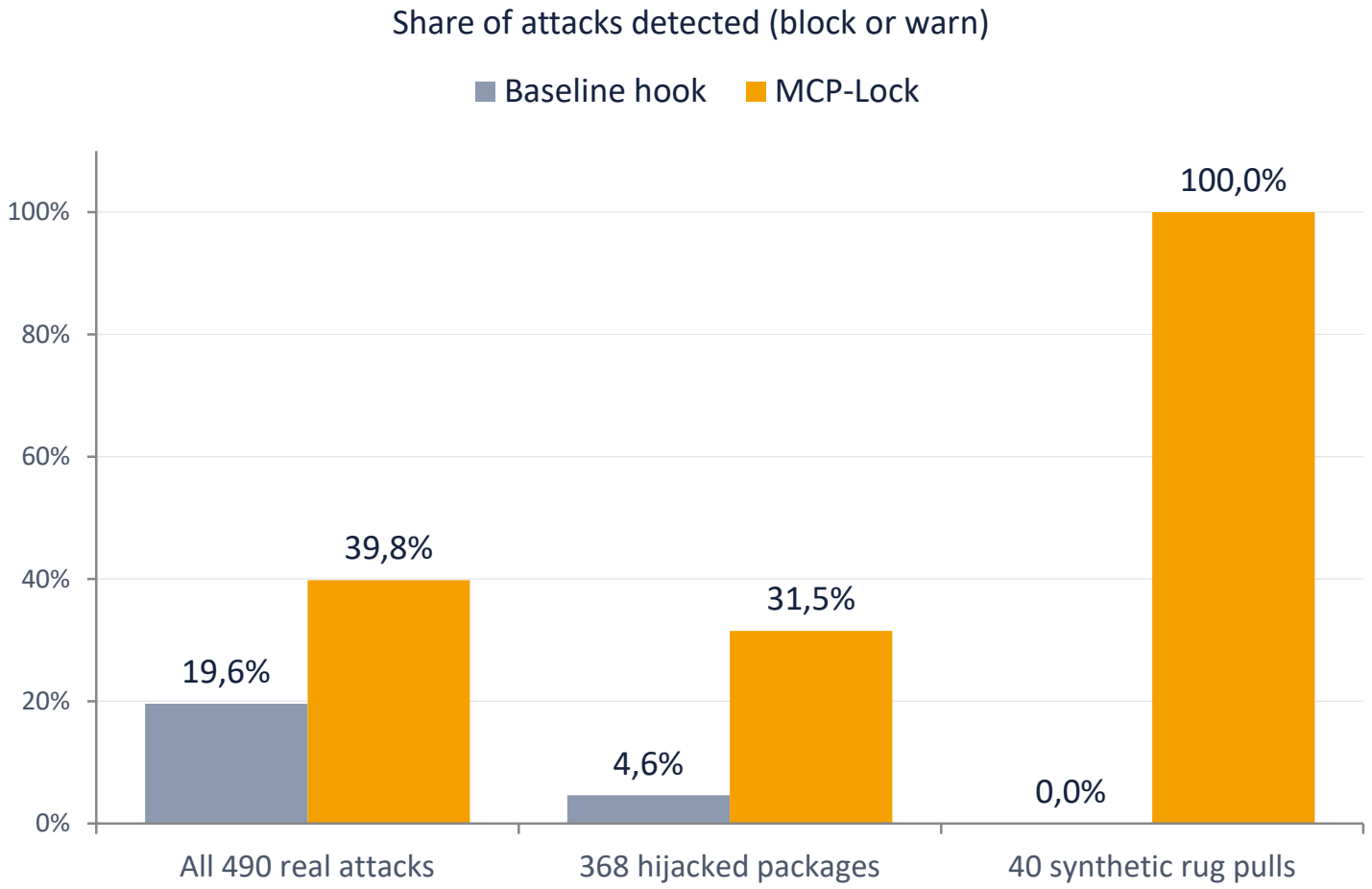
Tool definitions

The canonical hash of the tools differs from the lock; the diff is shown

WARN

Amber cards: added by MCP-Lock (compare a launch with its lock entry). Grey: the existing pre-install hook, kept as is.

Result: twice as many real attacks caught



2×

detection: 39.8% vs 19.6%

99 vs 0

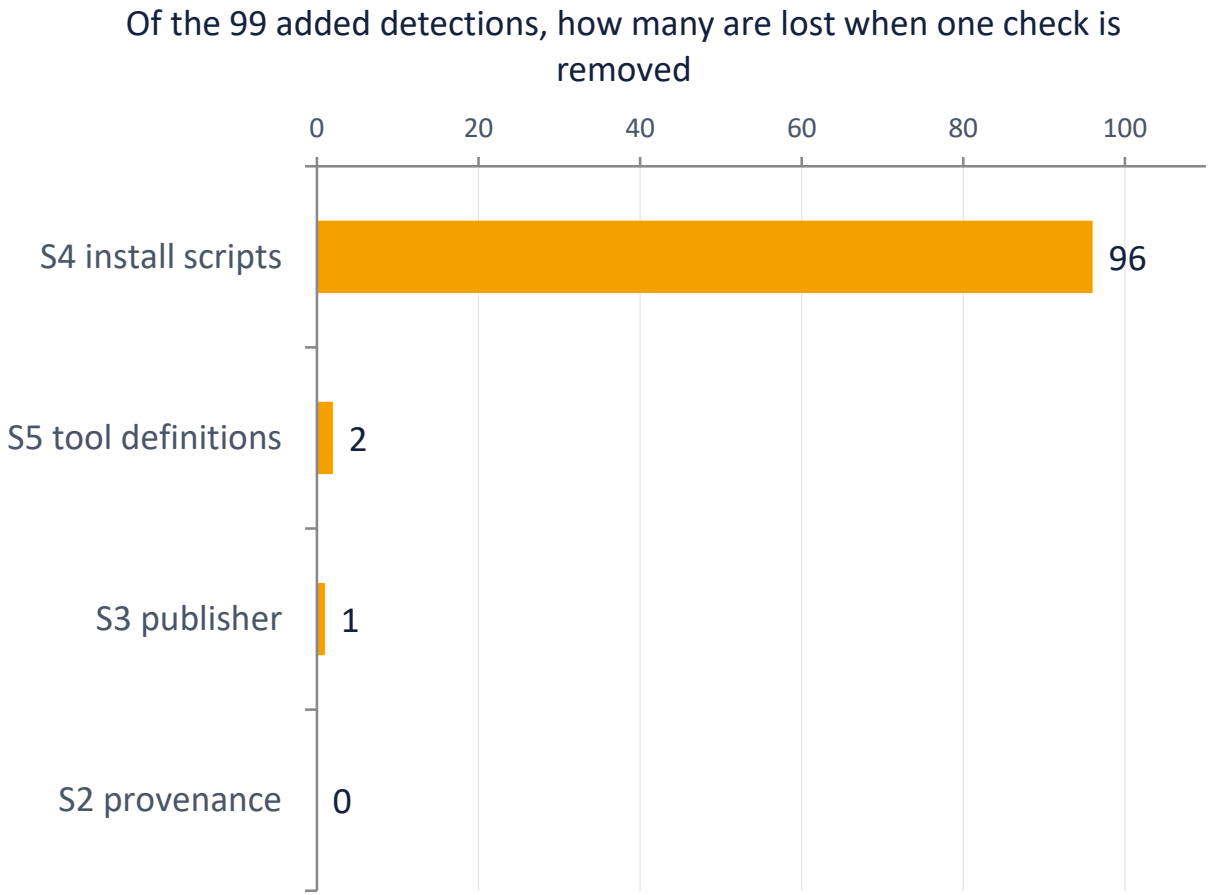
attacks only MCP-Lock caught vs only the baseline caught ($p = 3.2 \times 10^{-30}$)

40 / 40

tool-definition rug pulls caught; the baseline caught 0

Pre-registered test (H1), exact McNemar with Holm correction. Real attacks: 22 baseline-paper scenarios + 468 malicious npm/PyPI versions (OpenSSF, Datadog).

Where the extra detections come from



Install-script diffs do most of the work

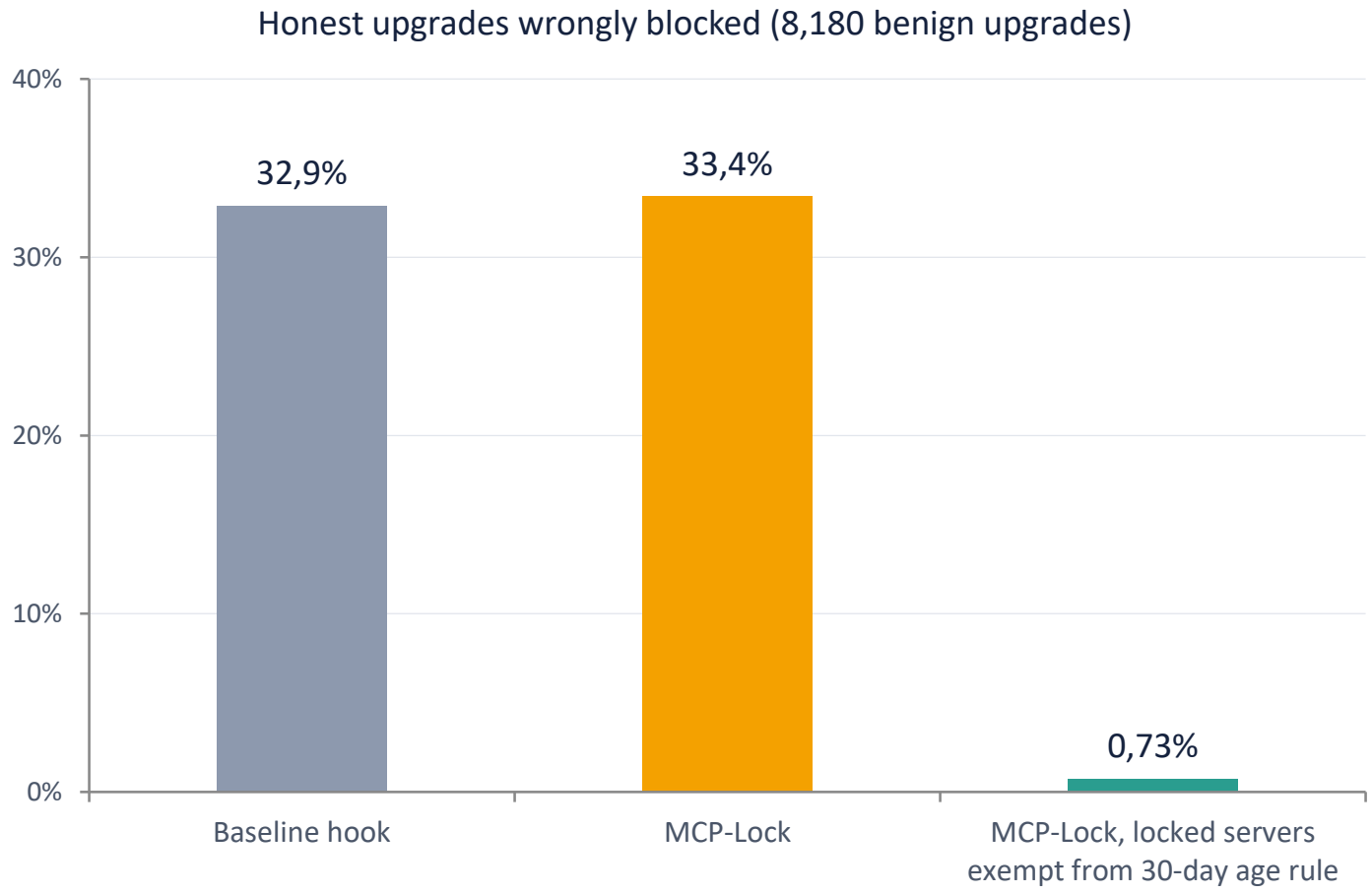
Hijacked packages usually add an install script. A lock that remembers “this server had none” turns that into a block.

Not just one worm campaign

Outside both Shai-Hulud waves, MCP-Lock still detects **32.0%** of malicious versions vs **21.2%** for the baseline ($p = 1.5\times10^{-11}$, exploratory).

Ablation on the pre-registered corpus. S5 is the only check that sees rug pulls in tool descriptions (R-S: 40/40).

The cost: almost no extra false blocks



+0.42 pp

extra false blocks over the baseline: small but statistically significant (H2)

0.73%

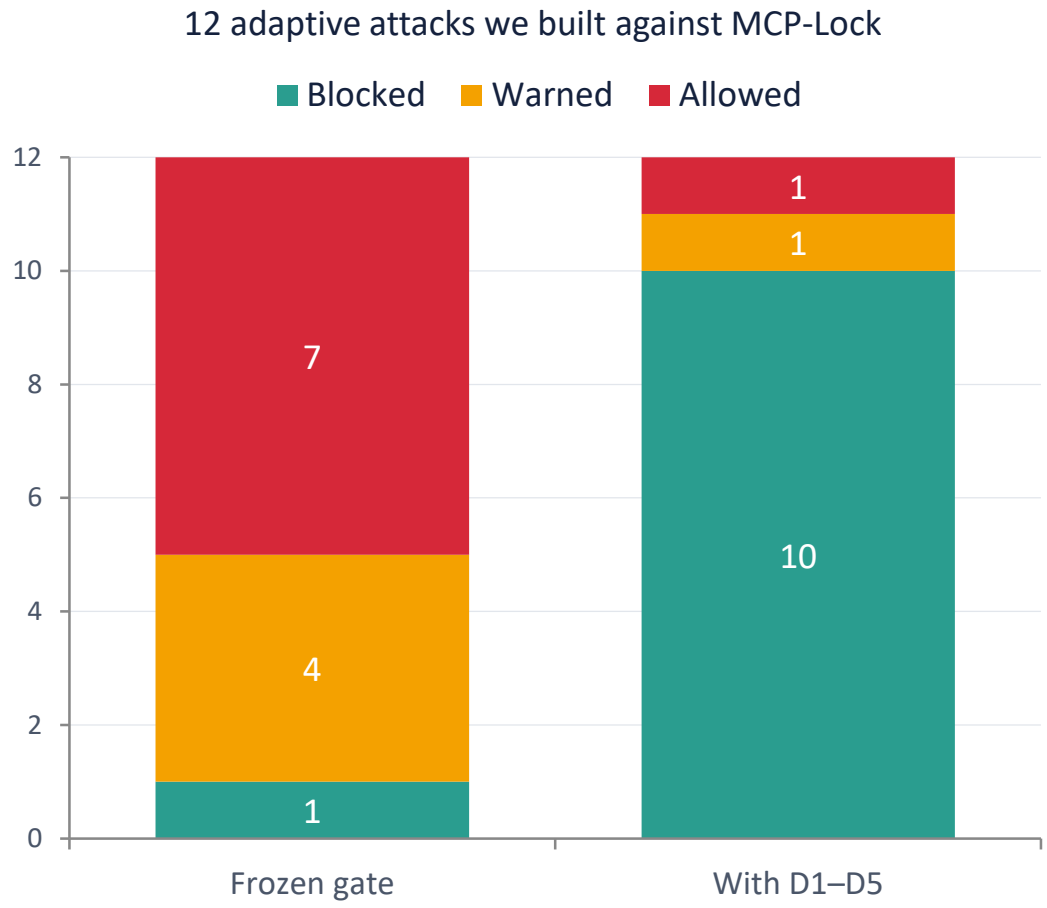
false blocks once locked servers skip the baseline's age rule, which causes almost all of them

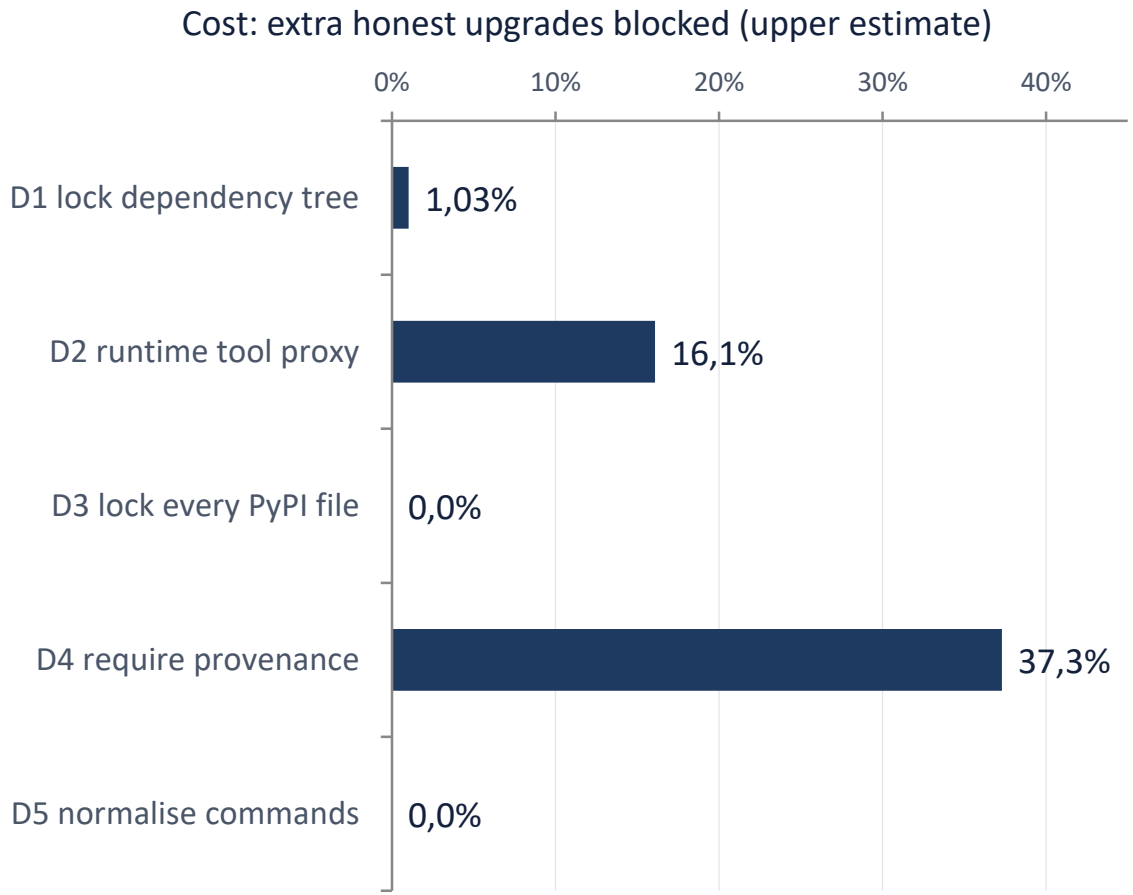
16.5%

of honest upgrades get a warning to review, mostly changed tool definitions

Age-rule exemption is an exploratory sensitivity analysis, not part of the pre-registered test.

Adaptive attacks, and how we close the gaps





Still open: a payload shipped through the genuine release workflow (allowed) and a new, unlike name (warned). Hardening is exploratory, not pre-registered.

Built to be trusted, and to be practical

OSF

Pre-registered

Hypotheses, data and a kill rule fixed on OSF before any scored run (DOI 10.17605/OSF.IO/2AMJ3)

p

Sound statistics

Exact McNemar tests with Holm correction; Wilson 95% intervals on every rate

11/11

Checked live

Attack servers run in hardened Docker: runtime verdicts match the analysis in 11 of 11

→

Safe suggestions

A verified safe version is offered for 114/116 hijacked cases and 40/40 rug pulls; none was unsafe

78.6%

Beats simple fixes

Exact pinning stops 78.6% of hijacks but freezes all 8,180 honest upgrades

103

Open and tested

Open-source code, replay bundles and 103 automated tests on GitHub

Takeaways

1

Real problem

36.5% of MCP packages change their tools within 90 days, and 97.0% of those changes are invisible to metadata checks

2

Clear improvement

2× the real attacks caught (39.8% vs 19.6%), every rug pull caught, at +0.42 pp false blocks

3

Honest limits

Adaptive attacks pass the frozen gate 7 of 12 times; five switches close all but two, each with a measured cost



Thank you. Questions?

Code: github.com/ftaakash/MCP-Lock · Pre-registration: osf.io/2amj3